

COMPLETE TUTORIAL · STEP-BY-STEP GUIDE

Open WebUI Tor, I2P & Stealth Web Scraper

A three-path web scraping infrastructure for local AI:
Clearnet, Tor (Snowflake) and I2P, all in Docker.

Repository

github.com/thotganesh/openwebui-tor-i2p-scraper

Project author: thotganesh

Stack: Docker Compose · Playwright · FastAPI · Tor/Snowflake · i2pd · Open WebUI

Generated on October 1, 2026 from the current state of the main branch (17 commits)

Table of Contents

- 1. Introduction 3**
 - 1.1 The project at a glance 3
 - 1.2 How to read this guide 3
- 2. Core features 4**
 - 2.1 The three paths compared 4
 - 2.2 Why Chromium for Clearnet and Firefox for Tor/I2P 4
- 3. Architecture 5**
 - 3.1 What happens when the AI asks for a page 5
 - 3.2 Ports and services 6
- 4. Prerequisites 6**
- 5. Quick Start 6**
- 6. Step-by-step tutorial 8**
 - 6.1 Prepare the folders 8
 - 6.2 tor-proxy: Tor + Snowflake 8
 - 6.3 browser-tor: browsing through Tor 10
 - 6.4 browser-clear: normal browsing with stealth 15
 - 6.5 browser-i2p: Garlic Routing with I2P 16
 - 6.6 docker-compose.yml 18
 - 6.7 Build and startup 19
 - 6.8 Verification: the test battery 19
 - 6.9 Integration with Open WebUI 20
 - 6.10 Smart text extraction 22
- 7. Security and OpSec 23**
- 8. Troubleshooting 23**
 - 8.1 Tor never reaches 100% 24
- 9. Useful commands 24**
- 10. Known issues and suggested improvements 25**
- 11. Project history 25**
 - 11.1 Main milestones 26
- Appendix A · Glossary 27**
 - References 27

1. Introduction

Open WebUI Tor, I2P & Stealth Web Scraper is a Docker-based infrastructure that gives a local AI model (the kind you run with Open WebUI) the ability to **read the web** in three different ways: over the regular internet (Clearnet), over the **Tor** anonymity network, and over **I2P**. Each path is an independent container running a real browser (Playwright) that loads the page, extracts the useful content and hands it back to the model as **clean Markdown**.

The project grew out of practical problems you hit when connecting an LLM to the internet:

- **Heavy pages.** Images, fonts and videos eat RAM and, above all, add noise that costs tokens. Here media is blocked at the network level.
- **Anti-bot protections.** A plain `requests.get()` gets stopped (e.g. by Cloudflare). The Clearnet browser uses `playwright-stealth` to look like a real user.
- **Privacy and OpSec.** Anonymous browsing needs Tor and a browser configured not to leak anything (WebRTC, fingerprinting, geolocation...).
- **ISPs that block Tor.** The **Snowflake** transport routes Tor traffic through WebRTC volunteers, bypassing those blocks.
- **Alternative networks.** .i2p sites need an I2P router (`i2pd`) and a browser isolated from the regular network.
- **Cookie banners and paywalls.** They are removed before extraction, so the text reaching the model is just the content.

1.1 The project at a glance

Item	Value
Docker containers	5 (tor-proxy, browser-tor, browser-clear, i2p-proxy, browser-i2p)
Browsers / engines	Chromium + stealth (Clearnet), hardened Firefox (Tor), hardened Firefox (I2P)
Open WebUI tools	3 (browse_web, browse_tor, browse_i2p)
FastAPI services	3 <code>app.py</code> files, about 350 lines of Python in total
Commits on main	17 (from 28 to 30 September 2026)
Languages / technologies	Python, injected JavaScript, Docker, Go (to build Snowflake), shell

IMPORTANT

This document describes a technical browsing and privacy tool. Use it in compliance with the laws of your country and with the terms of service of the sites you visit. Anonymity is not impunity: it is up to you to use Tor and I2P for legitimate purposes (research, privacy, access to censored information).

1.2 How to read this guide

If you already know Docker, jump to **chapter 5 (Quick Start)**: a handful of commands is enough. If you want to understand and rebuild every piece from scratch, follow **chapter 6 (Step-by-step tutorial)**, where each file is shown in full, with the real code taken from the repository and an explanation of each choice. Chapters 7-9 cover security, troubleshooting and useful commands. Chapter 10 collects a few notes on the repository itself.

NOTE

The source files in the repository contain some comments and user-facing strings in Italian (the original language of the author). Code listings in this document are reproduced exactly as they appear in the repository, so those comments are left untouched.

2. Core features

Feature	Description
Triple browsing paths	Three independent browsers: Clearnet, Tor and I2P, each in its own container.
Dynamic OpSec & JS control	The Tor and I2P tools expose a <code>js_enabled</code> parameter. The AI can switch JavaScript off on request, reducing fingerprinting and attack surface.
Cookie / GDPR killer	Injected JavaScript detects and removes consent banners (CMPs) and some paywalls before extracting the text.
Smart extraction	Mozilla Readability + <code>html2text</code> return only the article as Markdown, with a fallback for home pages and search engines.
Token & RAM optimized	Images, videos and fonts are blocked through Playwright routing (<code>block_media</code>).
New Tor circuit on demand	Before each navigation, stem sends the NEWNYM signal to Tor's ControlPort.
Snowflake transport	Tor reaches the network through Snowflake (WebRTC/ICE), useful when the ISP blocks Tor.
Secrets out of the code	The ControlPort password and hash are read from the <code>.env</code> file and never written to the repository.

2.1 The three paths compared

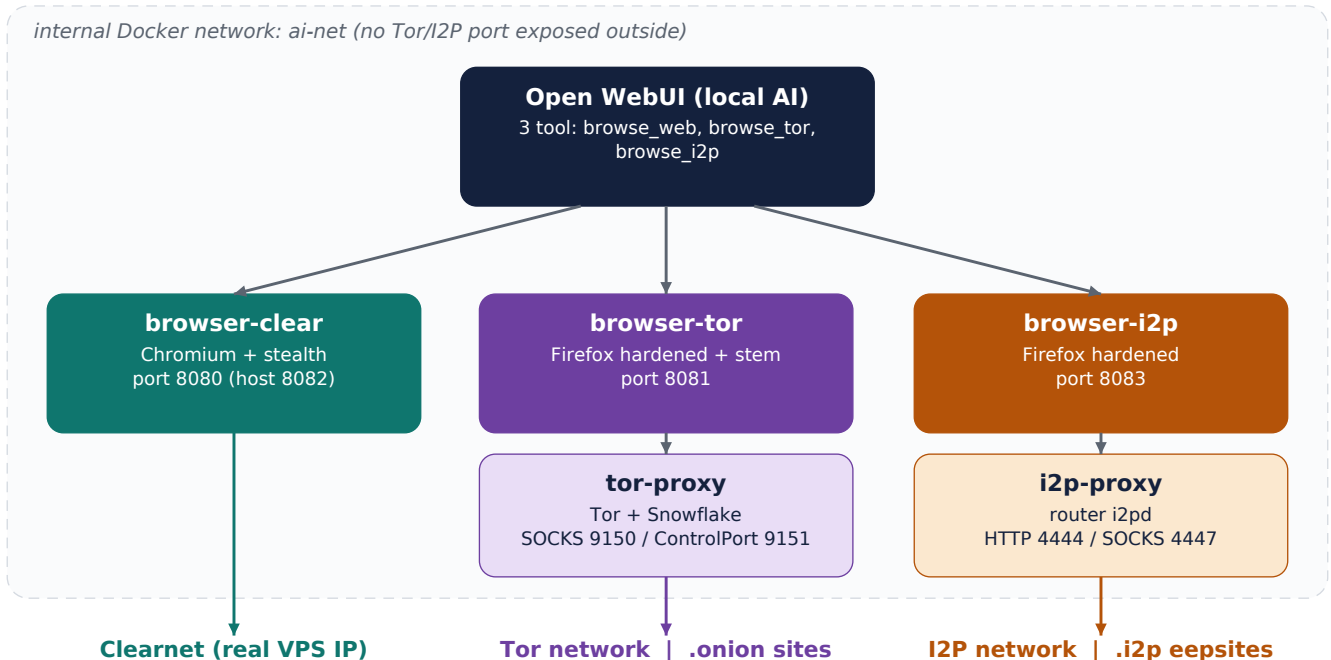
	browser-clear	browser-tor	browser-i2p
Engine	Chromium + playwright-stealth	Hardened Firefox	Hardened Firefox
Network	Clearnet (real IP)	Tor via Snowflake	I2P via i2pd (Garlic Routing)
Proxy used	none	<code>socks5://tor-proxy:9150</code>	<code>http://i2p-proxy:4444</code>
Internal port / host	8080 / 127.0.0.1:8082	8081 / 127.0.0.1:8081	8083 / 127.0.0.1:8083
Request parameters	<code>url</code> , <code>block_media</code>	<code>url</code> , <code>block_media</code> , <code>js_enabled</code>	<code>url</code> , <code>block_media</code> , <code>js_enabled</code>
WebRTC	on (like a real Chrome)	off (anti-leak)	off
Navigation timeout	60 s	90 s	120 s
Tool timeout	90 s	120 s	150 s
Error handling	HTTP 500 exception	HTTP 500 exception	"airbag": response with a clear message

2.2 Why Chromium for Clearnet and Firefox for Tor/I2P

On the regular internet the goal is to look like an ordinary user, so the project uses Chromium with stealth patches and WebRTC enabled (a real Chrome always has it). On Tor and I2P the goal is the opposite: **to be indistinguishable and leak nothing**. Firefox offers `privacy.resistFingerprinting` (the same technology as Tor Browser) and lets Playwright set the hardening preferences directly.

3. Architecture

All services live on the internal Docker network `ai-net`, shared with the Open WebUI container. The browsers are exposed on the host only on `127.0.0.1`, while the Tor and I2P proxies publish no ports at all.



3.1 What happens when the AI asks for a page

1. The user asks something in chat; the model decides which tool to call (`browse_web`, `browse_tor` or `browse_i2p`) based on the system prompt.
2. The tool (a small Python file inside Open WebUI) sends a POST to `/browse` on the matching container, passing the URL and options.
3. The FastAPI service launches a headless browser with the right proxy (none, Tor SOCKS5, I2P HTTP). For Tor it first requests a new circuit.
4. Playwright loads the page up to `domcontentloaded`, waits 2.5 seconds, and runs the cookie killer if JavaScript is enabled.
5. The HTML is cleaned with Readability and converted to Markdown with `html2text`. If Readability finds no article, the whole page is converted.
6. The service returns `title`, `final_url` and `text`; the tool truncates the text to 4000 characters and passes it to the model.

3.2 Ports and services

Service	Internal port	Exposed on host	Notes
tor-proxy	9150 (SOCKS), 9151 (ControlPort)	no	Never publish these ports
browser-tor	8081	127.0.0.1:8081	Endpoints: /browse, /health, /webrtc-leak-check
browser-clear	8080	127.0.0.1:8082	Endpoints: /browse, /health
i2p-proxy	4444 (HTTP), 4447 (SOCKS)	no	i2pd router, data in ./i2p-data
browser-i2p	8083	127.0.0.1:8083	Endpoint: /browse

NOTE

Inside Docker the services talk on their **internal** ports (browser-clear:8080), not on the ones mapped to the host (8082). The Open WebUI tools must therefore use the internal ports.

4. Prerequisites

- **A VPS with Docker and Docker Compose** (the `docker compose` command).
- **Outgoing UDP open.** Snowflake negotiates over WebRTC/ICE, which uses UDP. If your provider blocks it, Tor stays stuck in bootstrap.
- **A Docker network shared with Open WebUI**, called `ai-net` here. If it does not exist: `docker network create ai-net`.
- **Free ports on the host:** 8081 (browser-tor), 8082 (browser-clear), 8083 (browser-i2p).
- **Open WebUI already running** and attached to `ai-net`.
- **A model with function calling:** without it, the AI cannot call the tools.
- About **2 GB** of disk space for the Playwright image, plus the time to compile Snowflake on the first build.

5. Quick Start

For people who already know Docker: the whole stack in seven steps.

Installation commands

```
# 1) Clone the repository
git clone https://github.com/thotganesh/openwebui-tor-i2p-scraper.git
cd openwebui-tor-i2p-scraper

# 2) Create the shared Docker network (skip if it already exists)
docker network create ai-net

# 3) Connect Open WebUI to the network (skip if it's already connected)
docker network connect ai-net open-webui

# 4) I2P data folder: i2pd runs as non-root and needs write access
mkdir -p i2p-data && chmod -R 777 i2p-data

# 5) Secrets: copy .env.example to .env and fill in your values
cp .env.example .env
# Generate the hash of the ControlPort password:
docker run --rm debian:bookworm-slim bash -c \
  "apt-get update -qq && apt-get install -y -qq tor >/dev/null && tor --hash-password 'YourSecurePassword'"
# In .env: TOR_CONTROL_PASSWORD = plain-text password, TOR_CONTROL_HASH = generated hash

# 6) Build and start everything
docker compose up -d --build

# 7) Wait for Tor: look for "Bootstrapped 100% (done)"
docker compose logs -f tor-proxy
```

IMPORTANT

Change the default password. The `.env.example` file ships with the placeholder value `change-me`: do not leave it. After startup, **i2pd** needs 3-5 minutes on its first boot before `.i2p` sites resolve. The first build takes a few minutes (Snowflake compilation + a Playwright image of about 2 GB).

Once it is running, install the three tools in Open WebUI (section 6.9) and run the test battery (section 6.8).

6. Step-by-step tutorial

In this chapter we rebuild the stack from scratch. Every code block is the real content of the file in the repository.

6.1 Prepare the folders

Creating the structure

```
mkdir -p ~/ai-stack/{tor-snowflake,browser-tor,browser-clear,browser-i2p,i2p-data}
# VITAL: i2pd runs as non-root and needs permission to write its addressbook
chmod -R 777 ~/ai-stack/i2p-data
cd ~/ai-stack
```

Final project structure

```
~/ai-stack/
├── docker-compose.yml
├── .env                                (created from .env.example, must NOT be committed)
├── tor-snowflake/
│   ├── Dockerfile
│   └── torrc
├── browser-tor/
│   ├── Dockerfile
│   ├── app.py
│   ├── entrypoint.sh                (optional self-check)
│   └── startup_check.py             (optional self-check)
├── browser-clear/
│   ├── Dockerfile
│   └── app.py
├── browser-i2p/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
├── openwebui-tools/
│   ├── web_browser_clear.py
│   ├── web_browser_tor.py
│   └── web_browser_i2p.py
└── i2p-data/                        (i2pd router data, ignored by git)
```

NOTE

The `.gitignore` file excludes `.env`, `i2p-data/` and `__pycache__/`: secrets and router data never end up on GitHub.

6.2 tor-proxy: Tor + Snowflake

This container is the bridge to the Tor network. It uses **Snowflake** as a pluggable transport: the client contacts a broker through domain fronting and then negotiates over WebRTC with a volunteer acting as a proxy. To the ISP it looks like ordinary WebRTC traffic.

6.2.1 Multi-stage Dockerfile

tor-snowflake/Dockerfile

```
# --- Stage 1: build snowflake-client ---
FROM golang:1.23-bookworm AS builder

# Permette a Go di scaricare automaticamente la toolchain richiesta dal go.mod
ENV GOTOOCHAIN=auto

RUN git clone https://git.torproject.org/pluggable-transport/snowflake.git /tmp/snowflake \
    && cd /tmp/snowflake/client \
    && go build -o /usr/local/bin/snowflake-client .

# --- Stage 2: immagine finale, solo runtime ---
FROM debian:bookworm-slim

RUN apt-get update && apt-get install -y \
    tor obfs4proxy ca-certificates \
```

```
&& rm -rf /var/lib/apt/lists/*

COPY --from=builder /usr/local/bin/snowflake-client /usr/local/bin/snowflake-client
RUN chmod +x /usr/local/bin/snowflake-client

EXPOSE 9150
CMD ["tor", "-f", "/etc/tor/torrc"]
```

- **Stage 1 (builder):** compiles snowflake-client from source with Go.
- **Stage 2 (runtime):** starts from debian:bookworm-slim, installs only tor, obfs4proxy and the certificates, and copies the compiled binary. The final image stays lightweight.

NOTE

The Snowflake repository requires a very recent version of Go. With goLang:1.23 + GOTOOLCHAIN=auto, Go downloads the toolchain required by go.mod on its own.

6.2.2 ControlPort password

The ControlPort is used to ask Tor for a new circuit (the NEWNYM signal) before each request. It is password-protected: Tor wants the **hash**, the browser wants the **plain-text** password.

Generating the hash

```
docker run --rm debian:bookworm-slim bash -c \
  "apt-get update -qq && apt-get install -y -qq tor >/dev/null && tor --hash-password 'YourSecurePassword'"
# The last printed line is the hash, like: 16:AF97...
```

.env.example → copy it to .env and fill it in

```
# Copy to .env and fill in. NEVER commit .env
TOR_CONTROL_PASSWORD=change-me
TOR_CONTROL_HASH=16:PASTE_THE_GENERATED_HASH_HERE
```

WARNING

Never commit the .env file. The docker-compose.yml uses the \${VAR:?message} syntax: if a variable is missing, docker compose stops with an explicit error instead of starting with an empty password.

6.2.3 torrc

tor-snowflake/torrc

```
SocksPort 0.0.0.0:9150
UseBridges 1

ClientTransportPlugin snowflake exec /usr/local/bin/snowflake-client \
  -url https://snowflake-broker.torproject.net.global.prod.fastly.net/ \
  -front cdn.sstatic.net \
  -ice stun:stun.l.google.com:19302,stun:stun.antisp.com:3478,stun:stun.bluesip.net:3478,stun:stun.dus.net:3478,s
  ↪ tun:stun.ipygi.com:3478,stun:stun.sonetel.com:3478,stun:stun.uls.co.za:3478,stun:stun.voipgate.com:3478,stun:s
  ↪ tun.voys.nl:3478

Bridge snowflake 192.0.2.3:80 2B280B23E1107BB62ABFC40DDCC8824814F80A72
↪ fingerprint=2B280B23E1107BB62ABFC40DDCC8824814F80A72
↪ url=https://snowflake-broker.torproject.net.global.prod.fastly.net/
↪ fronts=foursquare.com,github.githubassets.com ice=stun:stun.l.google.com:19302,stun:stun.antisp.com:3478,stun:s
↪ tun.bluesip.net:3478,stun:stun.dus.net:3478,stun:stun.ipygi.com:3478,stun:stun.sonetel.com:3478,stun:stun.uls.co
↪ .za:3478,stun:stun.voipgate.com:3478,stun:stun.voys.nl:3478 utls-imitate=hellorandomizedalpn

ControlPort 0.0.0.0:9151
# HashedControlPassword is now injected via docker-compose and .env
```

- SocksPort 0.0.0.0:9150: the browsers connect here (inside the Docker network only).
- UseBridges 1 + ClientTransportPlugin snowflake ...: Tor always goes through Snowflake.
- Bridge snowflake 192.0.2.3:80 ...: the address 192.0.2.3 is a placeholder from the official documentation; with Snowflake it does not matter, because the client uses the broker and then a volunteer proxy.
- ControlPort 0.0.0.0:9151: used by stem for NEWNYM.

- No HashedControlPassword in the file: it is injected at startup by the compose file (see 6.6).

WARNING

The `-ice` and `Bridge` lines are **a single line each**. In this PDF they are wrapped for layout reasons (the `↪` symbol marks a continuation): in the real file they must never be split, except where there is a backslash.

6.3 browser-tor: browsing through Tor

A FastAPI service that drives a headless Firefox through the SOCKS proxy of `tor-proxy`. Every request starts with a fresh Tor circuit.

browser-tor/Dockerfile

```
FROM mcr.microsoft.com/playwright/python:v1.48.0-jammy

WORKDIR /app
RUN apt-get update && apt-get install -y libxml2-dev libxslt1-dev python3-dev && rm -rf /var/lib/apt/lists/*
RUN pip install fastapi uvicorn httpx playwright==1.48.0 stem readability-lxml html2text lxml
COPY app.py /app/app.py

EXPOSE 8081
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8081"]
```

The base image is the official Playwright image for Python. The `playwright==1.48.0` pin matters: the package version must match the browsers already bundled in the image, otherwise you get the error *Executable doesn't exist*. The `libxml2-dev` and `libxslt1-dev` packages are needed to build `lxml` (a Readability dependency).

6.3.1 Full source of app.py

browser-tor/app.py

```
import os
import asyncio
import socket
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from stem import Signal
from stem.control import Controller
from readability import Document
import html2text

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    wait_selector: str | None = None
    extract_text: bool = True
    block_media: bool = True
    js_enabled: bool = True

TOR_PROXY = {"server": "socks5://tor-proxy:9150"}
CONTROL_PASSWORD = os.environ.get("TOR_CONTROL_PASSWORD")

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,
    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "privacy.resistFingerprinting.letterboxing": True,
    "privacy.spoof_english": 2,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
    "media.navigator.enabled": False,
}

FIREFOX_PREFS_WEBRTC_TEST = dict(FIREFOX_PREFS)
FIREFOX_PREFS_WEBRTC_TEST["media.peerconnection.enabled"] = True
```

```

WEBRTC_LEAK_SCRIPT = """
async () => {
  if (typeof RTCPeerConnection === "undefined") return "DISABLED";
  return new Promise((resolve) => {
    const ips = new Set();
    const pc = new RTCPeerConnection({iceServers: [{urls: "stun:stun.l.google.com:19302"}]});
    pc.createDataChannel("");
    pc.onicecandidate = (e) => {
      if (!e.candidate) { resolve([...ips]); pc.close(); return; }
      const match = e.candidate.candidate.match(/(\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d{1,3})/);
      if (match) ips.add(match[1]);
    };
    pc.createOffer().then(o => pc.setLocalDescription(o));
    setTimeout(() => { resolve([...ips]); pc.close(); }, 5000);
  });
}
"""

COOKIE_KILLER_JS = """
async () => {
  const KNOWN_CMP_SELECTORS = [
    '#onetrust-banner-sdk', '#onetrust-consent-sdk',
    '#cookiebot', '#CybotCookiebotDialog',
    '.qc-cmp2-container', '#qc-cmp2-container',
    '#truste-consent-track', '.truste_box_overlay',
    '.cc-window', '.fc-consent-root', '#cookie-notice',
    '.paywall', '[class*="cookie-banner"]', '[id*="cookie-consent"]'
  ];
  KNOWN_CMP_SELECTORS.forEach(sel => {
    document.querySelectorAll(sel).forEach(el => el.remove());
  });
  document.body.style.overflow = 'auto';
}
"""

def request_new_circuit():
  try:
    tor_proxy_ip = socket.gethostbyname("tor-proxy")
    with Controller.from_port(address=tor_proxy_ip, port=9151) as controller:
      controller.authenticate(password=CONTROL_PASSWORD)
      controller.signal(Signal.NEWNYM)
      print(f"[info] Nuovo circuito richiesto con successo (via {tor_proxy_ip})")
  except Exception as e:
    print(f"[warn] Impossibile richiedere nuovo circuito: {e}")

def pulisci_testo(html_content: str) -> str:
  h = html2text.HTML2Text()
  h.ignore_links = False
  h.ignore_images = True
  h.body_width = 0

  try:
    doc = Document(html_content)
    testo_pulito = h.handle(doc.summary()).strip()
    if testo_pulito:
      return testo_pulito
  except Exception:
    pass

  try:
    return h.handle(html_content).strip()
  except Exception as e:
    return f"Errore nell'estrazione del testo: {e}"

@app.post("/browse")
async def browse(req: BrowseRequest):
  request_new_circuit()
  await asyncio.sleep(1)

  async with async_playwright() as p:
    browser = await p.firefox.launch(
      headless=True,
      proxy=TOR_PROXY,
      firefox_user_prefs=FIREFOX_PREFS,
    )
    context = await browser.new_context(
      viewport={"width": 1000, "height": 900},
      timezone_id="UTC",
      locale="en-US",
      java_script_enabled=req.js_enabled,
    )

```

```

page = await context.new_page()

if req.block_media:
    await page.route("**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}", lambda route: route.abort())

await page.goto(req.url, wait_until="domcontentloaded", timeout=90000)

await page.wait_for_timeout(2500)

# Only execute the Cookie Killer if JS is enabled!
if req.js_enabled:
    try:
        await page.evaluate(COOKIE_KILLER_JS)
    except Exception as e:
        print(f"[warn] Failed to execute Cookie Killer: {e}")

result = {"title": await page.title(), "final_url": page.url}
if req.extract_text:
    result["text"] = pulisci_testo(await page.content())

await browser.close()
return result

@app.get("/webrtc-leak-check")
async def webrtc_leak_check():
    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=TOR_PROXY,
            firefox_user_prefs=FIREFOX_PREFS_WEBRTC_TEST,
        )
        page = await (await browser.new_context()).new_page()
        await page.goto("data:text/html,<html><body></body></html>")
        leaked_ips = await page.evaluate(WEBRTC_LEAK_SCRIPT)
        await browser.close()
        return {
            "note": "Test con WebRTC volutamente riabilitato. In produzione (/browse) resta sempre disattivato.",
            "leaked_ips": leaked_ips,
            "leak_detected": len(leaked_ips) > 0,
        }

@app.get("/health")
async def health():
    return {"status": "ok"}

```

6.3.2 Block-by-block explanation

Block	What it does
BrowseRequest	Pydantic model of the request: <i>url</i> , <i>wait_selector</i> (not used in the flow), <i>extract_text</i> , <i>block_media</i> and <i>js_enabled</i> , all with safe defaults.
TOR_PROXY, CONTROL_PASSWORD	SOCKS5 address of the Tor container and the ControlPort password read from the environment variable (it comes from .env).
FIREFOX_PREFS	Firefox hardening: see table 6.3.3.
WEBRTC_LEAK_SCRIPT	Diagnostic JavaScript: tries to create an RTCPeerConnection and collects the IPs of the ICE candidates. If WebRTC is disabled it answers "DISABLED".
COOKIE_KILLER_JS	Removes from the DOM the containers of the most common CMPs (OneTrust, Cookiebot, Quantcast, TrustArc...) and restores scrolling.
request_new_circuit()	Resolves <i>tor-proxy</i> to an IP (stem needs a literal IP), authenticates on ControlPort 9151 and sends Signal.NEWNYM. On failure it prints a warning but does not block navigation.
pulisci_testo()	Two-level extraction: Readability.summary() → html2text; if empty, html2text on the whole HTML.
POST /browse	New circuit → 1 s pause → Firefox with proxy and prefs → context with 1000x900 viewport, UTC timezone and en-US locale → optional media blocking → goto (domcontentloaded, 90 s) → 2.5 s wait → cookie killer (only if JS is on) → extraction.

Block	What it does
GET /webrtc-leak-check	Diagnostic endpoint: launches Firefox with WebRTC deliberately re-enabled and shows which IPs would leak. It demonstrates why it must stay off in production.
GET /health	Returns {"status": "ok"} for reachability checks.

6.3.3 Firefox hardening preferences

Preference	Effect
media.peerconnection.enabled = False	Disables WebRTC: prevents the real IP from leaking via ICE/STUN around the SOCKS proxy.
network.dns.disablePrefetch = True	No DNS prefetch (requests that could leave the intended path).
network.prefetch-next = False	No preloading of the next links.
geo.enabled = False	Geolocation disabled.
privacy.resistFingerprinting = True	Normalizes values such as resolution, timezone and user agent, reducing the browser's uniqueness.
...resistFingerprinting.letterboxing = True	The window uses rounded dimensions (browser-tor only).
privacy.spoof_english = 2	Forces English language (browser-tor only).
webgl.disabled = True	WebGL off: it is a source of GPU fingerprinting.
dom.battery.enabled = False	No access to battery status.
device.sensors.enabled = False	Device sensors not accessible.
media.navigator.enabled = False	No camera/microphone API (browser-tor only).

6.3.4 Why a new circuit on every request

Playwright does not support username/password authentication on SOCKS5, so the "random credentials = isolated circuit" technique is not available. The project uses the ControlPort instead: NEWNYM asks Tor to use new circuits for subsequent connections. Tor rate-limits the signal to roughly one every 10 seconds, so two back-to-back requests may share the same exit IP.

6.3.5 Startup self-check (optional)

The repository contains two extra files, `entrypoint.sh` and `startup_check.py`, designed as a "safety belt": before starting the service they verify that WebRTC is really disabled and, if they detect a leak, **refuse to start the container**.

browser-tor/entrypoint.sh

```
#!/bin/bash
set -e

echo "[entrypoint] Attendo che tor-proxy sia raggiungibile..."
for i in $(seq 1 30); do
    if curl -s -m 2 --socks5-hostname tor-proxy:9150 https://check.torproject.org/api/ip > /dev/null 2>&1; then
        echo "[entrypoint] tor-proxy raggiungibile."
        break
    fi
    sleep 2
done

python3 /app/startup_check.py
if [ $? -ne 0 ]; then
    echo "[entrypoint] Self-check fallito. Uscita."
    exit 1
fi
```

```
echo "[entrypoint] Self-check superato, avvio uvicorn..."
exec uvicorn app:app --host 0.0.0.0 --port 8081
```

browser-tor/startup_check.py

```
import asyncio
import sys
from playwright.async_api import AsyncPlaywright

TOR_PROXY = {"server": "socks5://tor-proxy:9150"}

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,
    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "privacy.resistFingerprinting.letterboxing": True,
    "privacy.spoof_english": 2,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
    "media.navigator.enabled": False,
}

WEBRTC_LEAK_SCRIPT = """
async () => {
    if (typeof RTCPeerConnection === "undefined") return "DISABLED";
    return new Promise((resolve) => {
        const ips = new Set();
        const pc = new RTCPeerConnection({iceServers: [{urls: "stun:stun.l.google.com:19302"}]});
        pc.createDataChannel("");
        pc.onicecandidate = (e) => {
            if (!e.candidate) { resolve([...ips]); pc.close(); return; }
            const match = e.candidate.candidate.match(/(\\d{1,3}\\.(\\d{1,3}\\.(\\d{1,3}\\.(\\d{1,3})?)/);
            if (match) ips.add(match[1]);
        };
        pc.createOffer().then(o => pc.setLocalDescription(o));
        setTimeout(() => { resolve([...ips]); pc.close(); }, 5000);
    });
}
"""

async def main():
    print("[startup-check] Verifica configurazione WebRTC di produzione...", flush=True)
    async with AsyncPlaywright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=TOR_PROXY,
            firefox_user_prefs=FIREFOX_PREFS,
        )
        page = await (await browser.new_context()).new_page()
        await page.goto("data:text/html,<html><body></body></html>")
        result = await page.evaluate(WEBRTC_LEAK_SCRIPT)
        await browser.close()

    if result == "DISABLED":
        print("[startup-check] OK: RTCPeerConnection non disponibile, nessun rischio leak.", flush=True)
        sys.exit(0)

    if isinstance(result, list) and len(result) > 0:
        print(f"[startup-check] ERRORE CRITICO: WebRTC leak rilevato! IP esposti: {result}", flush=True)
        print("[startup-check] Il container NON si avvia per motivi di sicurezza.", flush=True)
        sys.exit(1)

    print("[startup-check] OK: nessun leak rilevato.", flush=True)
    sys.exit(0)

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

In the current Dockerfile the CMD starts uvicorn directly and these two files are not copied into the image: they are a ready-to-use addition but **not yet wired in**. Enabling them takes three changes, described in chapter 10.

6.4 browser-clear: normal browsing with stealth

Here the goal is the opposite of Tor: **look like an ordinary user** and get past anti-bot protections, using your real IP.

browser-clear/Dockerfile

```
FROM mcr.microsoft.com/playwright/python:v1.48.0-jammy

WORKDIR /app
RUN apt-get update && apt-get install -y libxml2-dev libxslt1-dev python3-dev && rm -rf /var/lib/apt/lists/*
RUN pip install fastapi uvicorn playwright-stealth httpx playwright==1.48.0 readability-lxml html2text lxml
COPY app.py /app/app.py

EXPOSE 8080
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
```

browser-clear/app.py

```
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from playwright_stealth import Stealth
from readability import Document
import html2text

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    wait_selector: str | None = None
    extract_text: bool = True
    block_media: bool = True

COOKIE_KILLER_JS = """
async () => {
    const KNOWN_CMP_SELECTORS = [
        '#onetrust-banner-sdk', '#onetrust-consent-sdk',
        '#cookiebot', '#CybotCookiebotDialog',
        '.qc-cmp2-container', '#qc-cmp2-container',
        '#truste-consent-track', '.truste_box_overlay',
        '.cc-window', '.fc-consent-root', '#cookie-notice',
        '.paywall', '[class*="cookie-banner"]', '[id*="cookie-consent"]'
    ];
    KNOWN_CMP_SELECTORS.forEach(sel => {
        document.querySelectorAll(sel).forEach(el => el.remove());
    });
    document.body.style.overflow = 'auto';
}
"""

def pulisci_testo(html_content: str) -> str:
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0

    try:
        # Tentativo 1: Cerca un articolo strutturato
        doc = Document(html_content)
        testo_pulito = h.handle(doc.summary()).strip()
        if testo_pulito:
            return testo_pulito
    except Exception:
        pass

    try:
        # Fallback: Se è una home page/motore di ricerca, formatta tutto
        return h.handle(html_content).strip()
    except Exception as e:
        return f"Errore nell'estrazione del testo: {e}"

@app.post("/browse")
async def browse(req: BrowseRequest):
    async with Stealth().use_async(async_playwright()) as p:
        browser = await p.chromium.launch(headless=True)
        context = await browser.new_context(
            user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)"
        )
```

```

        ↪ Chrome/126.0 Safari/537.36",
        viewport={"width": 1366, "height": 768},
    )
    page = await context.new_page()

    if req.block_media:
        await page.route("**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}", lambda route: route.abort())

    await page.goto(req.url, wait_until="domcontentloaded", timeout=60000)

    await page.wait_for_timeout(2500)
    await page.evaluate(COOKIE_KILLER_JS)

    result = {"title": await page.title(), "final_url": page.url}
    if req.extract_text:
        result["text"] = pulisci_testo(await page.content())

    await browser.close()
    return result

@app.get("/health")
async def health():
    return {"status": "ok"}

```

- **Stealth:** `Stealth().use_async(async_playwright())` (the playwright-stealth 2.x API) masks the typical automation signals.
- **Realistic user agent and viewport:** Chrome 126 on Windows, 1366x768.
- **WebRTC left on:** a real Chrome always has it, and here the real IP is not a secret.
- **block_media on by default** for speed and RAM savings. With `false` it loads everything like a real user: useful against aggressive anti-bot systems, because a Chrome that never loads images is anomalous behavior.

6.5 browser-i2p: Garlic Routing with I2P

I2P is an anonymity network separate from Tor, with its own sites (*eepsites*, domain `.i2p`) reachable only from inside. The `i2pd` router builds the tunnels; the browser uses it as an HTTP proxy on port 4444.

browser-i2p/Dockerfile

```

FROM mcr.microsoft.com/playwright/python:v1.40.0-jammy
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN playwright install firefox
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8083"]

```

browser-i2p/requirements.txt

```

fastapi
uvicorn
playwright
readability-lxml
html2text
pydantic

```

browser-i2p/app.py

```

import asyncio
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from readability import Document
import html2text

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    extract_text: bool = True

```

```

    block_media: bool = True
    js_enabled: bool = True

# i2pd espone il proxy HTTP sulla porta 4444
I2P_PROXY = {"server": "http://i2p-proxy:4444"}

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,
    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
}

COOKIE_KILLER_JS = """
async () => {
    const KNOWN_CMP_SELECTORS = [
        '#onetrust-banner-sdk', '.cc-window', '.qc-cmp2-container',
        '[class*="cookie-banner"]', '[id*="cookie-consent"]'
    ];
    KNOWN_CMP_SELECTORS.forEach(sel => {
        document.querySelectorAll(sel).forEach(el => el.remove());
    });
    document.body.style.overflow = 'auto';
}
"""

def pulisci_testo(html_content: str) -> str:
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0
    try:
        doc = Document(html_content)
        testo = h.handle(doc.summary()).strip()
        if testo: return testo
    except Exception:
        pass
    try:
        return h.handle(html_content).strip()
    except Exception as e:
        return f"Errore estrazione: {e}"

@app.post("/browse")
async def browse(req: BrowseRequest):
    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=I2P_PROXY,
            firefox_user_prefs=FIREFOX_PREFS,
        )
        context = await browser.new_context(
            viewport={"width": 1000, "height": 900},
            timezone_id="UTC",
            locale="en-US",
            java_script_enabled=req.js_enabled,
        )
        page = await context.new_page()

        if req.block_media:
            await page.route("**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}", lambda route: route.abort())

        # --- AIRBAG: Blocco try/except per gestire i siti offline o I2P non sincronizzato ---
        try:
            await page.goto(req.url, wait_until="domcontentloaded", timeout=120000)
            await page.wait_for_timeout(2500)
        except Exception as e:
            await browser.close()
            return {
                "title": "Errore di Rete I2P",
                "final_url": req.url,
                "text": f"Sito I2P attualmente irraggiungibile o router I2P non ancora sincronizzato. Dettaglio: {e}"
            }

        # Se il JS è abilitato, puliamo i popup
        if req.js_enabled:
            try:

```

```

        await page.evaluate(COOKIE_KILLER_JS)
    except Exception as e:
        print(f"[warn] Failed to execute Cookie Killer: {e}")

    result = {"title": await page.title(), "final_url": page.url}
    if req.extract_text:
        result["text"] = pulisci_testo(await page.content())

    await browser.close()
    return result

```

- **Garlic Routing:** all traffic goes through the i2p-proxy container, never through your real IP.
- **JavaScript control:** `js_enabled=false` disables JS entirely, recommended on I2P against fingerprinting and hostile scripts.
- **"Airbag":** the try/except around `page.goto` handles offline eepsites (very common) by returning an understandable message instead of a 500 error.
- **Long timeout (120 s):** I2P is intrinsically slow. The Open WebUI tool uses 150 s and recommends `block_media=true`.

6.6 docker-compose.yml

docker-compose.yml

```

services:
  tor-proxy:
    build: ./tor-snowflake
    container_name: tor-proxy
    networks: [ai-net]
    environment:
      - TOR_CONTROL_HASH=${TOR_CONTROL_HASH:?Set TOR_CONTROL_HASH in .env}
    command: ["sh", "-c", "exec tor -f /etc/tor/torrc --HashedControlPassword \"$$TOR_CONTROL_HASH\""]
    volumes:
      - ./tor-snowflake/torrc:/etc/tor/torrc:ro
    restart: unless-stopped

  browser-tor:
    build: ./browser-tor
    container_name: browser-tor
    networks: [ai-net]
    environment:
      - TOR_CONTROL_PASSWORD=${TOR_CONTROL_PASSWORD:?Set TOR_CONTROL_PASSWORD in .env}
    depends_on: [tor-proxy]
    ports:
      - "127.0.0.1:8081:8081"
    restart: unless-stopped

  browser-clear:
    build: ./browser-clear
    container_name: browser-clear
    networks: [ai-net]
    ports:
      - "127.0.0.1:8082:8080"
    restart: unless-stopped

  i2p-proxy:
    image: purplei2p/i2pd:latest
    container_name: i2p-proxy
    networks: [ai-net]
    command: --httpproxy.address=0.0.0.0 --socksproxy.address=0.0.0.0
    restart: unless-stopped
    volumes:
      - ./i2p-data:/home/i2pd/data

  browser-i2p:
    build: ./browser-i2p
    container_name: browser-i2p
    networks: [ai-net]
    depends_on: [i2p-proxy]
    ports:
      - "127.0.0.1:8083:8083"
    restart: unless-stopped

networks:
  ai-net:

```

```
external: true
```

- **tor-proxy command:** `exec tor -f /etc/tor/torrc --HashedControlPassword "$$TOR_CONTROL_HASH"`. The double \$\$ makes a single \$ reach the container's shell, which reads the variable at startup.
- **Required variables:** `${VAR:?...}` makes compose fail if `TOR_CONTROL_HASH` or `TOR_CONTROL_PASSWORD` are missing.
- **ai-net is external: true:** the network already exists and is shared with Open WebUI.
- **Ports on 127.0.0.1:** the browsers are reachable only from the host itself. Never publish Tor's ports (9150/9151) or the I2P proxy ports.
- **restart: unless-stopped** on every service: the stack comes back on its own after a VPS reboot.
- **depends_on:** browser-tor waits for tor-proxy, browser-i2p waits for i2p-proxy (start order only; it does not guarantee the network is already ready).

6.7 Build and startup

Starting the stack

```
cd ~/ai-stack
docker compose build
docker compose up -d
docker compose ps          # all five containers "Up", no restart loop
docker compose logs -f tor-proxy # wait for "Bootstrapped 100% (done)"
```

IMPORTANT

I2P warm-up: the i2pd router needs about 3-5 minutes on its very first boot to build its tunnels and populate the addressbook. Test `http://stats.i2p` only after that.

6.8 Verification: the test battery

Run these seven tests in order. Always use real URLs: a placeholder URL returns a 404 that looks like a code problem.

Seven verification tests

```
# 1) Tor's SOCKS works and the IP is anonymous
docker run --rm --network ai-net curlimages/curl -s --socks5-hostname tor-proxy:9150
↳ https://check.torproject.org/api/ip

# 2) browser-clear: must show IsTor:false with the VPS's real IP
curl -s -X POST http://127.0.0.1:8082/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 3) browser-tor: must show IsTor:true with an exit node IP
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 4) browser-tor on a real .onion (DuckDuckGo)
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "http://duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion"}'

# 5) Circuit rotation: NEWNYM has a rate limit of ~10 s
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'; echo; sleep 12
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 6) WebRTC diagnostic test (WebRTC deliberately re-enabled: must show the real IP leak)
curl -s http://127.0.0.1:8081/webrtc-leak-check

# 7) browser-i2p on an eepsite, JavaScript OFF (wait ~5 min after first boot)
curl -s -X POST http://127.0.0.1:8083/browse -H "Content-Type: application/json" \
  -d '{"url": "http://stats.i2p", "js_enabled": false}'
```

Test	Expected result
Direct SOCKS	{"IsTor":true,"IP":"..."}
browser-clear	IsTor: false with the VPS's real IP
browser-tor	IsTor: true, exit node IP different from the VPS's
browser-tor on .onion	HTTP 200 with the page's title and text
Circuit rotation	Different exit IPs between the two requests
webrtc-leak-check	leaked_ips containing the real IP: proof that WebRTC must stay disabled
browser-i2p on stats.i2p	HTTP 200 with the eepsite's title and text (after warm-up)

6.9 Integration with Open WebUI

6.9.1 Network and reachability

Check that Open WebUI can reach the browsers

```
docker inspect open-webui --format '{{json .NetworkSettings.Networks}}' # ai-net must appear
docker network connect ai-net open-webui # only if missing
docker exec open-webui curl -s -m 10 http://browser-clear:8080/health # {"status":"ok"}
docker exec open-webui curl -s -m 10 http://browser-tor:8081/health # {"status":"ok"}
```

6.9.2 Installing the three tools

1. In Open WebUI go to **Admin → Workspace → Tools → new tool**.
2. Paste the content of **one file at a time** (one tool per file) from the openwebui-tools/ folder.
3. Save and enable **all three** tools in your chat; make sure your model supports function calling.

IMPORTANT

All three tools are required: they are the three pillars of the same stack. The Clear tool talks to browser-clear, the Tor tool to browser-tor and the I2P tool to browser-i2p.

Tool 1 · web_browser_clear.py

```
"""
title: Web Browser (Clear)
description: Naviga siti web normali tramite browser headless Chromium con stealth anti-detection
"""
import httpx

class Tools:
    def __init__(self):
        self.base_url = "http://browser-clear:8080"

    async def browse_web(self, url: str) -> str:
        """
        Naviga una pagina web normale ed estrae il testo.
        :param url: URL completo da visitare (es. https://example.com)
        """
        async with httpx.AsyncClient(timeout=90) as client:
            r = await client.post(f"{self.base_url}/browse", json={"url": url})
            data = r.json()
            return f"Titolo: {data.get('title')}\nURL finale: {data.get('final_url')}\n\n{data.get('text',
↳ ''')[0:4000]}"
```

Tool 2 · web_browser_tor.py

```
"""
title: Web Browser (Tor/Anonymous)
description: Naviga siti .onion o normali in forma completamente anonima tramite rete Tor (Snowflake). Permette il
↳ controllo del JavaScript per azzerare il fingerprinting.
"""

import httpx

class Tools:
    def __init__(self):
        self.base_url = "http://browser-tor:8081"

    async def browse_tor(
        self, url: str, js_enabled: bool = True, block_media: bool = True
    ) -> str:
        """
        Naviga una pagina web (anche .onion) tramite la rete Tor per anonimato completo.
        :param url: URL completo da visitare, incluso indirizzi .onion.
        :param js_enabled: Imposta a False per disabilitare completamente JavaScript. Da usare quando l'utente
        ↳ chiede la MASSIMA SICUREZZA e anti-fingerprinting.
        :param block_media: Imposta a True per bloccare immagini/video pesanti e velocizzare Tor.
        """
        async with httpx.AsyncClient(timeout=120) as client:
            try:
                r = await client.post(
                    f"{self.base_url}/browse",
                    json={
                        "url": url,
                        "js_enabled": js_enabled,
                        "block_media": block_media,
                    },
                )
                r.raise_for_status()
                data = r.json()

                # Creiamo l'etichetta visiva per capire subito se JS era acceso o spento
                status_js = (
                    "JS: OFF (Stealth/Anonymized)" if not js_enabled else "JS: ON"
                )
                text_content = data.get("text", "")[:4000]

                return f"[TOR NETWORK | {status_js}]\nTitolo: {data.get('title')}\nURL finale: "
                ↳ {data.get('final_url')}\n\n{text_content}"
            except Exception as e:
                return f"Errore durante la navigazione Tor: {str(e)}"
```

Tool 3 · web_browser_i2p.py

```
"""
title: Web Browser (I2P Darknet)
description: Naviga esclusivamente la rete chiusa I2P (siti .i2p) in totale sicurezza. Controllo JavaScript
↳ integrato per massima OpSec.
"""

import httpx

class Tools:
    def __init__(self):
        # Punta alla nuova porta 8083 del container browser-i2p
        self.base_url = "http://browser-i2p:8083"

    async def browse_i2p(
        self, url: str, js_enabled: bool = True, block_media: bool = True
    ) -> str:
        """
        Naviga siti della rete I2P (.i2p) tramite router i2pd locale (Garlic Routing).
        :param url: URL completo da visitare (deve essere un indirizzo .i2p).
        :param js_enabled: Imposta a False per disabilitare JavaScript. Consigliato per la massima sicurezza
        ↳ contro fingerprinting e script ostili sulla rete I2P.
        :param block_media: Imposta a True per bloccare immagini/video pesanti e velocizzare la navigazione I2P
        ↳ (che è intrinsecamente lenta).
        """
        async with httpx.AsyncClient(timeout=150) as client:
            try:
                r = await client.post(
                    f"{self.base_url}/browse",
```

```

        json={
            "url": url,
            "js_enabled": js_enabled,
            "block_media": block_media,
        },
    )
    r.raise_for_status()
    data = r.json()

    status_js = "JS: OFF (Stealth)" if not js_enabled else "JS: ON"
    text_content = data.get("text", "")[:4000]

    return f"[I2P NETWORK | {status_js}]\nTitolo: {data.get('title')}\nURL finale:
↳ {data.get('final_url')}\n\n{text_content}"
except Exception as e:
    return f"Errore durante la navigazione I2P (Il sito potrebbe essere offline, molto comune su I2P):
↳ {str(e)}"

```

NOTE

The [:4000] limit on the text keeps token usage under control: raise it if you need longer extracts. Each function's docstring is essential: Open WebUI uses it to tell the model what the tool does and what the parameters are for.

6.9.3 Avoiding conflicts with native Web Search

If Open WebUI's built-in Web Search is enabled, turn it off (**Admin Panel** → **Settings** → **Web Search** → **OFF**): with competing tools, small models choose inconsistently. Then put this in the global System Prompt:

Recommended system prompt

To browse normal websites ALWAYS use the browse_web tool.
 For .onion sites, or when the user explicitly requests anonymity, ALWAYS use the browse_tor tool.
 For .i2p eepsites ALWAYS use the browse_i2p tool.
 You have no other way to access the internet.
 Keep JavaScript enabled by default; disable it (js_enabled=false) only when the user asks for maximum security.

If the model does not call the tools, check that they are enabled in the chat and that the model has the function-calling capability (model settings → Capabilities).

6.10 Smart text extraction

Four mechanisms, present in all the browsers:

1. **Resource blocking** (block_media): `page.route("**/*.{png,jpg,...}")` stops images, fonts and videos before they download. Faster pages and less memory. On sites with serious anti-bot protection, turn it off.
2. **Cookie killer**: removes the containers of the main CMPs from the DOM and restores scrolling. It does not bypass server-side paywalls.
3. **Two-level extraction**: Readability for the structured article, then an `html2text` fallback on the whole HTML (home pages, indexes, search results). You never get a hard error.
4. `domcontentloaded` + **a fixed 2500 ms wait** instead of `networkidle`: ad-heavy sites never reach a "quiet network" and would time out.

WARNING

Some CMPs have anti-tamper protection: on `repubblica.it` (Iubenda), removing the banner with `el.remove()` emptied the whole page (the HTML went from 496,335 to 15 characters). In those cases it is better to **hide** the element with `el.style.setProperty('display', 'none', 'important')` instead of removing it.

7. Security and OpSec

The project's security choices, summarized in one place:

Area	Choice in the project	Why
Secrets	Password and hash in the .env file, ignored by git; \${VAR:?} in compose	No credentials in the repository; startup blocked if missing
Network exposure	Browser ports only on 127.0.0.1; Tor and I2P proxies publish no ports	Nobody from outside can use your anonymity proxies
WebRTC	Disabled on Tor and I2P; enabled only in the diagnostic test	With WebRTC on, the real IP leaks via ICE/STUN
Fingerprinting	resistFingerprinting, WebGL off, 1000x900 viewport, UTC, en-US	Makes the browser look like other Tor users
JavaScript	js_enabled parameter controllable by the AI	Maximum security on darknets when needed
Tor circuits	NEWNYM before every request	Reduces correlation between different requests
I2P isolation	Separate container, dedicated proxy, no clearnet traffic	No exposure of the real IP
Self-check	startup_check.py blocks the container if it detects a WebRTC leak	Fail-closed: better not to start than to leak the IP

TIP

The `chmod -R 777` on the `i2p-data` folder is the quickest way to let `i2pd` (a non-root user) write. On a shared server consider a more restrictive alternative: give the folder to the UID used by the container instead of granting write permission to everyone.

NOTE

Page content comes from untrusted sources. A site can contain text designed to manipulate the model (*prompt injection*). Treat the text returned by the tools as data, not as instructions, especially if your model can perform other actions.

8. Troubleshooting

Error / symptom	Cause	Solution
I2P: container Restarting (139)	Permission denied on /home/i2pd/data	<code>chmod -R 777 i2p-data</code>
I2P: NS_ERROR_UNKNOWN_PROXY_HOST	The i2p-proxy container is down or unreachable	Check permissions on i2p-data, then <code>docker compose restart i2p-proxy</code>
I2P: Host not found in addressbook	The router is still warming up	Wait ~5 minutes, then test <code>http://stats.i2p</code>
I2P tool returns an error / timeout	Eepsite offline (very common) or slow tunnels	Retry later or try another eepsite
Error parsing Bridge address 'auto'	Incorrect or leftover torrc syntax	Rewrite the torrc from scratch
general SOCKS server failure (obfs4)	obfs4 bridge with made-up fingerprint/cert	Use Snowflake with the official bridge line
invalid go version '1.25.12'	Image's Go too old	Multi-stage with golang and ENV <code>GOTOOLCHAIN=auto</code>

Error / symptom	Cause	Solution
pip3: not found	Node.js variant of the Playwright image	Use <code>mcr.microsoft.com/playwright/python</code>
No module named 'playwright'	The Python image doesn't include the package	Explicit <code>pip install playwright==1.48.0</code>
Executable doesn't exist ... chrome-headless-shell	Python package newer than the image's browsers	Pin <code>playwright==</code> to the image version
port is already allocated	Host port already in use	Change the mapping (e.g. <code>8082:8080</code>)
cannot import name 'stealth_async'	playwright-stealth 2.x has a new API	<code>Stealth().use_async(async_playwright())</code>
RTCPeerConnection is not defined	With WebRTC disabled the API doesn't exist	Run the leak test with WebRTC re-enabled (dedicated endpoint)
Browser does not support socks5 proxy authentication	Playwright doesn't support user/password on SOCKS5	Isolate circuits with ControlPort and NEWNYM
Invalid IP address: tor-proxy	stem requires a literal IP	<code>socket.gethostbyname("tor-proxy")</code>
Document is empty / Unparseable	Readability doesn't find an article	html2text fallback on the whole HTML
Page emptied after the cookie killer	The CMP detects the banner removal (anti-tamper)	Hide with <code>display:none</code> instead of <code>remove()</code>
Cannot read properties of null (reading 'style')	<code>document.body</code> absent during a reload	<code>if (document.body) + try/except</code>
Cookie banner still in the text	Case-sensitive selectors or CMP not in the list	Add the site's CMP IDs (or selectors with the <code>i</code> flag)
404 response in tests	Placeholder URL taken literally	Use a real URL
curl responds with 0 bytes	Error 500 hidden by <code>curl -s</code>	<code>curl -v</code> and <code>docker compose logs <service></code>

8.1 Tor never reaches 100%

If the tor-proxy logs stay stuck on bootstrap, the most likely cause is **outgoing UDP blocked** by the VPS provider (Snowflake requires it). Check your provider's firewall rules, then restart with `docker compose restart tor-proxy`.

9. Useful commands

Day-to-day stack management

```

cd ~/ai-stack
docker compose ps                # container status
docker compose logs -f browser-tor  # Tor browser logs in real time
docker compose logs -f i2p-proxy    # monitor I2P tunnels
docker compose logs browser-clear --tail 30 | grep -i warn

# after modifying an app.py / Dockerfile:
docker compose build browser-clear browser-tor browser-i2p && docker compose up -d browser-clear browser-tor
↵ browser-i2p

docker compose restart tor-proxy    # restart only Tor
docker compose restart i2p-proxy    # restart only I2P
docker compose down                 # stop everything (the ai-net network remains)

```

10. Known issues and suggested improvements

Reading the actual code next to the README surfaced a few differences and possible improvements. None is serious, but they are worth knowing about so that text and code stay consistent.

#	Observation	Suggestion
1	Self-check not active. <code>entrypoint.sh</code> and <code>startup_check.py</code> are in the repo but the browser-tor Dockerfile does not copy them and its CMD starts <code>uvicorn</code> directly.	Add to the Dockerfile: <code>COPY startup_check.py entrypoint.sh /app/, RUN chmod +x /app/entrypoint.sh</code> and CMD <code>["/app/entrypoint.sh"]</code> . The script uses <code>curl</code> : install it with <code>apt</code> .
2	/health on browser-i2p. The README (8.1) tests it, but <code>browser-i2p/app.py</code> does not define the endpoint.	Add <code>@app.get("/health")</code> as in the other two services, or remove the test from the README.
3	Cookie killer using <code>remove()</code>. The README warns that <code>remove()</code> empties some pages (e.g. <code>lubenda</code>), but the code still uses <code>el.remove()</code> in all three services.	Replace with <code>el.style.setProperty('display', 'none', 'important')</code> and restore scrolling.
4	Clear tool without <code>block_media</code>. In the README the tool has a <code>block_media</code> parameter; the <code>web_browser_clear.py</code> file does not (it uses the server default).	Align the file with the README so the blocking can be turned off on anti-bot sites.
5	Small Dockerfile differences. The README shows simplified versions (e.g. without <code>libxml2-dev</code> , with <code>EXPOSE 8083</code> on browser-i2p).	Copy the real Dockerfiles into the README, or link to the files.
6	Different Playwright versions. <code>browser-tor</code> and <code>browser-clear</code> use 1.48.0, <code>browser-i2p</code> uses the 1.40.0 image with an unpinned playwright in <code>requirements.txt</code> .	Pin <code>playwright==1.40.0</code> in <code>requirements.txt</code> (or align everything to 1.48.0) to avoid mismatches with the bundled browsers.
7	Mixed languages. Code comments and tool strings are in Italian, the README is in English.	Pick one language for code strings, or document the choice.

11. Project history

The project grew over two days with 17 commits. Here is the sequence, which also mirrors the logical order in which to build the stack.

#	Date	Commit
1	2026-09-28	First release: Tor Snowflake + Stealth Scraper infrastructure
2	2026-09-28	Initialize README for Open WebUI Tor & Stealth Scraper
3	2026-09-28	Add complete tutorial for Tor and Snowflake setup
4	2026-09-28	Fix formatting and enhance README content
5	2026-09-28	Add <code>web_browser_clear.py</code> for web browsing functionality
6	2026-09-28	Add Tor web browser tool for anonymous browsing
7	2026-09-28	Document Open WebUI tools and installation steps
8	2026-09-30	Revise README for I2P integration and tool updates
9	2026-09-30	Revise README.md for clarity and completeness

#	Date	Commit
10	2026-09-30	Add I2P web browser functionality
11	2026-09-30	Enhance browse_tor with JS and media control options
12	2026-09-30	Revise README for clarity and updated features
13	2026-09-30	Enhance README for I2P integration and features
14	2026-09-30	Add Quick Start section to README
15	2026-09-30	feat: Add I2P Garlic Routing and secure Tor credentials via .env
16	2026-09-30	Revise README for secrets configuration and Tor setup
17	2026-09-30	Clarify HashedControlPassword setup instructions

11.1 Main milestones

Phase	What was done
Sep 28 • Foundations	Tor + Snowflake infrastructure and stealth scraper; complete tutorial; first Open WebUI tools (Clear and Tor).
Sep 30 • Control and I2P	Tor tool enhanced with JavaScript and media control; I2P browser with Garlic Routing and its tool added.
Sep 30 • Security	ControlPort password and hash moved to .env; clear instructions on the hash; Quick Start section in the README.

Appendix A · Glossary

Term	Meaning
Open WebUI	Web interface for local AI models, extensible with Python tools.
Tool (function calling)	A function the model can call to obtain information or perform actions.
Playwright	Microsoft library to drive browsers (Chromium, Firefox, WebKit) from code.
Headless	A browser that runs without a graphical interface.
Tor	Circuit-based anonymity network (onion routing): traffic passes through several encrypted nodes.
Snowflake	A Tor pluggable transport that uses WebRTC volunteers as bridges, useful where Tor is blocked.
Domain fronting	A technique that reaches a service through a "cover" domain on a CDN.
WebRTC / ICE / STUN	Technologies for peer-to-peer connections in the browser; they can reveal the real IP.
ControlPort / NEWNYM	Tor's control interface; NEWNYM requests new circuits for subsequent connections.
stem	Python library to control Tor through the ControlPort.
.onion	Hidden services reachable only through Tor.
I2P / i2pd	Anonymity network separate from Tor; <i>i2pd</i> is the lightweight C++ router implementation.
Eepsite	A website hosted on I2P, domain .i2p.
Garlic Routing	A variant of onion routing used by I2P, bundling several messages ("cloves").
Fingerprinting	Identifying a user by the characteristics of their browser (resolution, fonts, GPU...).
CMP	Consent Management Platform: cookie and consent banners (OneTrust, Cookiebot...).
Readability	Mozilla algorithm that isolates the main content of a page.
html2text	Python library that converts HTML to Markdown.
OpSec	Operational Security: practices to avoid revealing sensitive information.
Prompt injection	Instructions hidden in web content to manipulate the model.

References

- Repository: <https://github.com/thotganesh/openwebui-tor-i2p-scraper>
- Tor Project and Snowflake: <https://www.torproject.org>
- I2P / i2pd: <https://i2pd.website>
- Playwright for Python: <https://playwright.dev/python>
- Open WebUI: <https://openwebui.com>